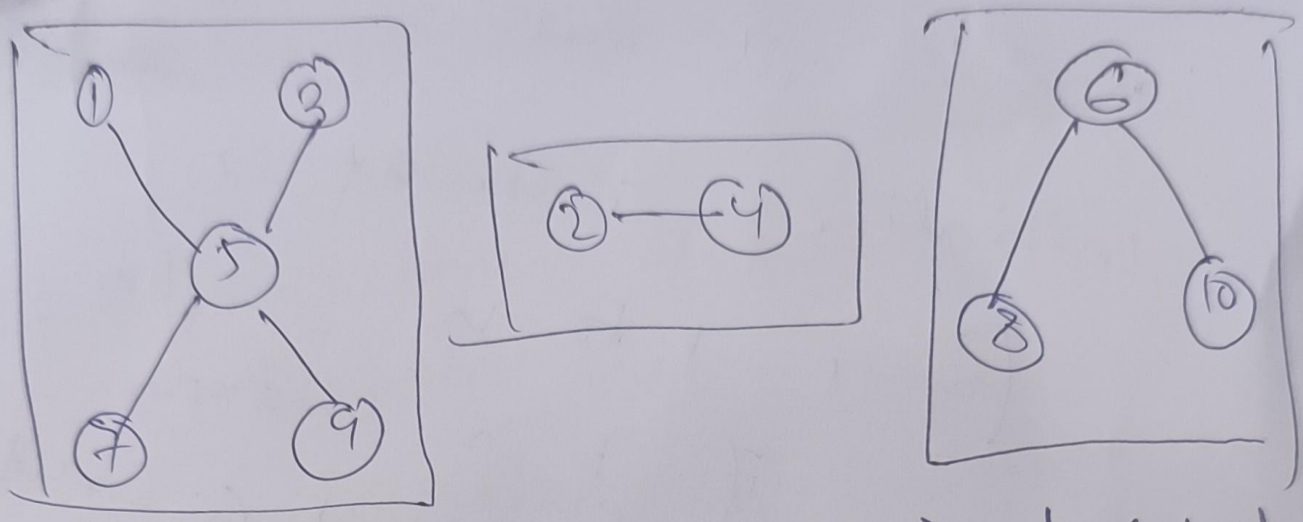
Connected Components.

Undirected graph $G=(V,E)$



Connected Components — Maximal subsets of reachable vertices (or equivalence class).

Applications

Detecting network failures

Genome clustering.

& many.

Connected Component in BFS

---

mark all vertices 1, 2, ..., n as unexplored.

Initialize CC Count = 0

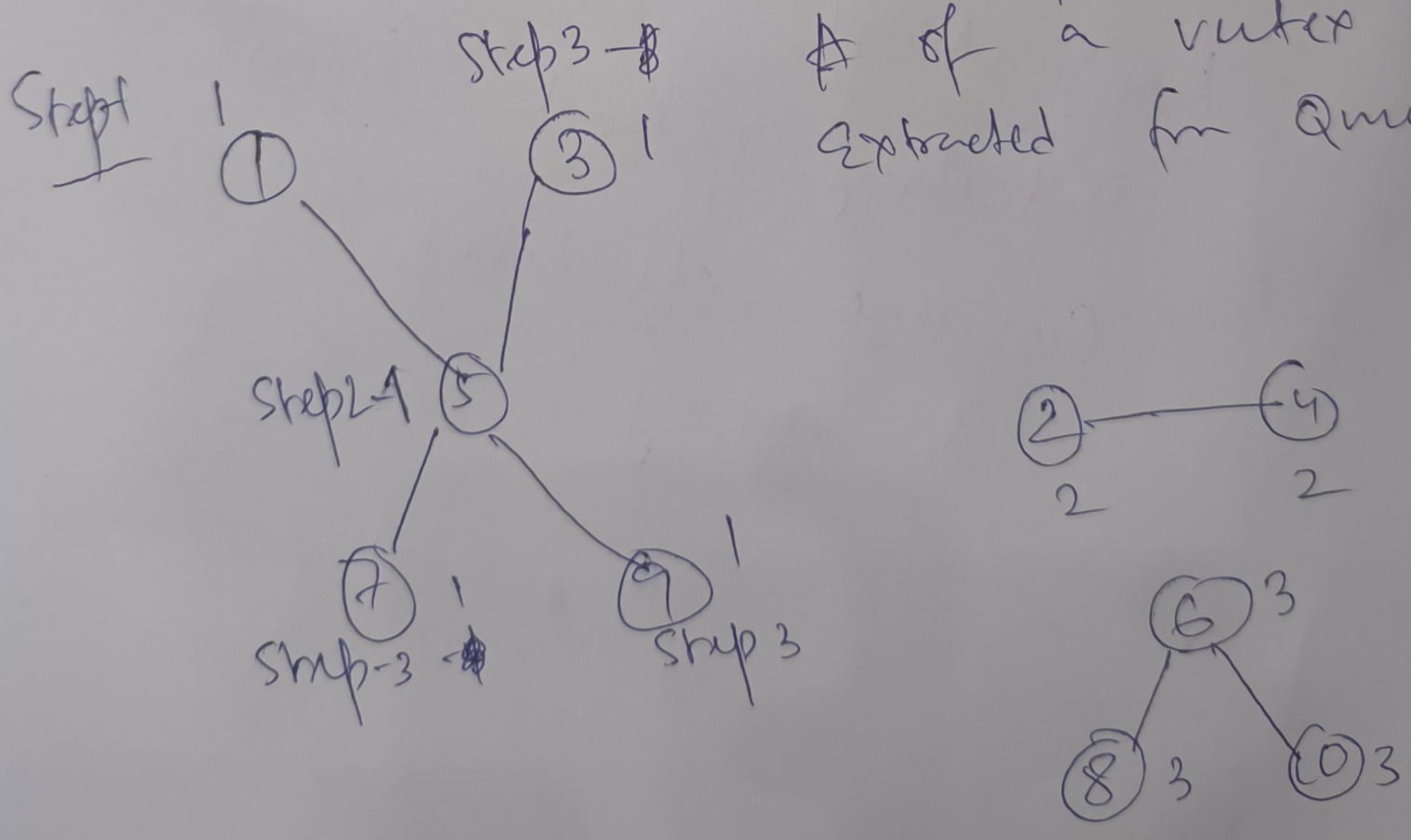
for i = 1 to n // try all vertices.

if i is unexplored

increment CC Count by 1

BFS (G, i)

{ While updating CC # of a vertex extracted from Queue



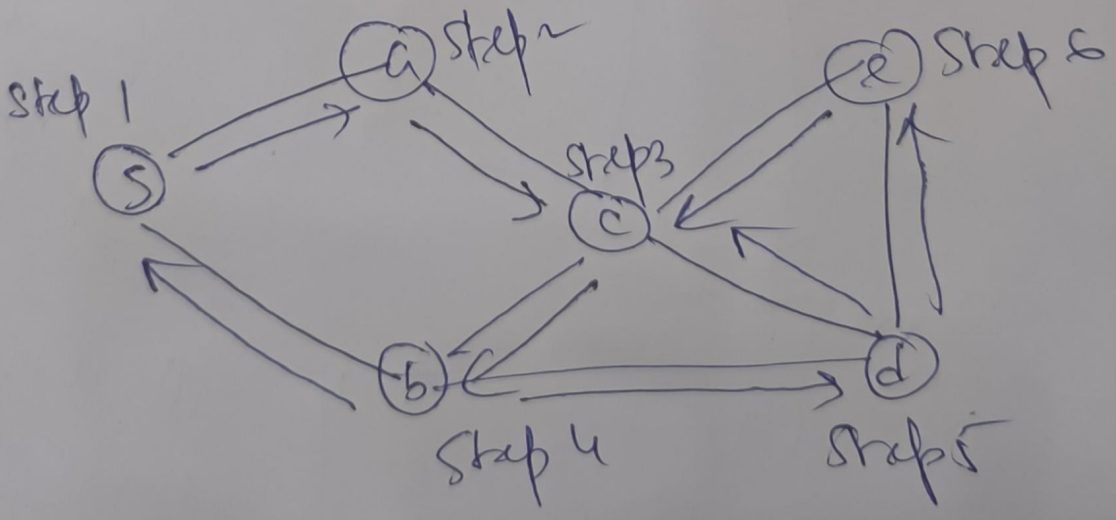
Claim :- (a) At termination.

$cc[u] = cc[v] \Leftrightarrow u$ & v in same Connected Component

Idea :- Correctness of BFS + Explored vertex is never marked unexplored

(b) Algo runs in $O(m+n)$ time

DFS



Iterative version

Mark all vertices as unexplored
S = a stack data structure (LIFO)
initialized with s

while S ≠ ∅

remove the top node of S
Say v. ("pop")

if v is unexplored
mark v as explored
for each (v,w) in
adj. list of v
add w to the
front of S
("push")

Recursive version

DFS (G,s) // all vertices unexplored before the call

mark s as Explored
for each (s,v) in the adj list
if v is unexplored of S
DFS (G,v).

Claim :- At the end of BFS

v is explored $\Leftrightarrow$ $\exists$ path for s to v in G

Reason :- DFS is a special case of Generic algorithm.

Claim 2 :- Running time $O(n_s + m_s)$

n_s → # reachable vertices

m_s → reachable edges.

Corollary :- DFS can compute Connected Components in linear time.

Applications

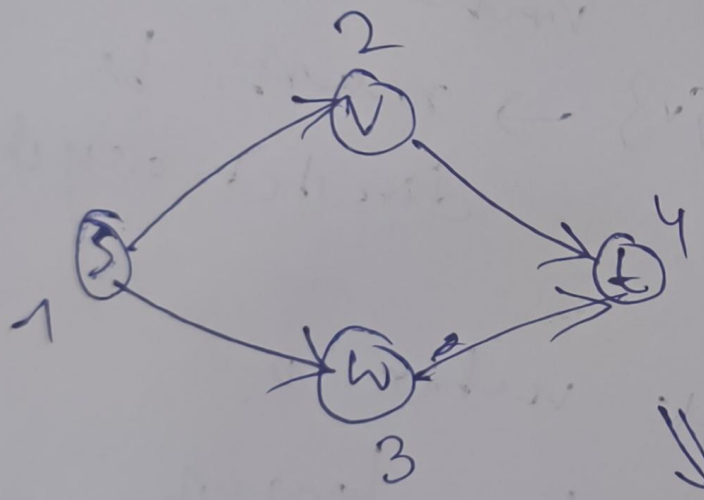
- Topological ordering
- Strongly Connected Component

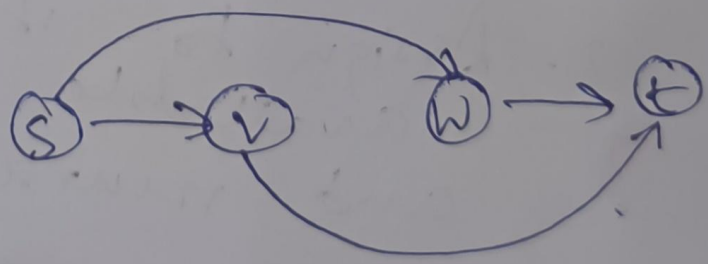
Topological Ordering

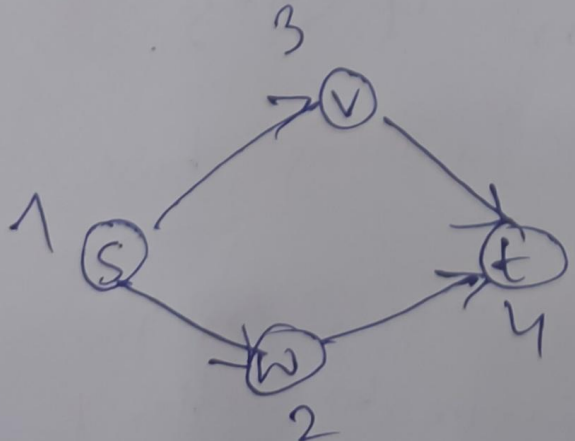
Directed graph $G = (V, E)$.

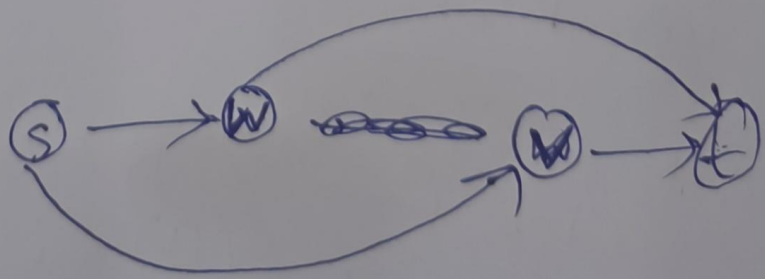
A labeling f of G is such that

* Unique $f(v) \in \{1, 2, \ldots, n\}$ for $\forall v \in V$.
* for every $(v, w) \in E$, $f(v) < f(w)$.









Thm:- Every directed acyclic graph has at least one "topological ordering".

Lemma - Every directed acyclic graph has at least one "sink".

A vertex with no outgoing edges

Proof:- Assign $f[v] = n$ to sink vertex v (exists!).

Recurse on $G \setminus \{v\}$ → must be directed acyclic ∎

ALG

1. Find a sink vertex v.
2. Assign to it the largest available level.
   and recurse on $G \setminus \{v\}$.

Correctness

If $f[v] = i$, no edge from v to vertices with $f(v) < i$.

Running time

$O(m^2)$.

[Can we do better for sparse graphs?]

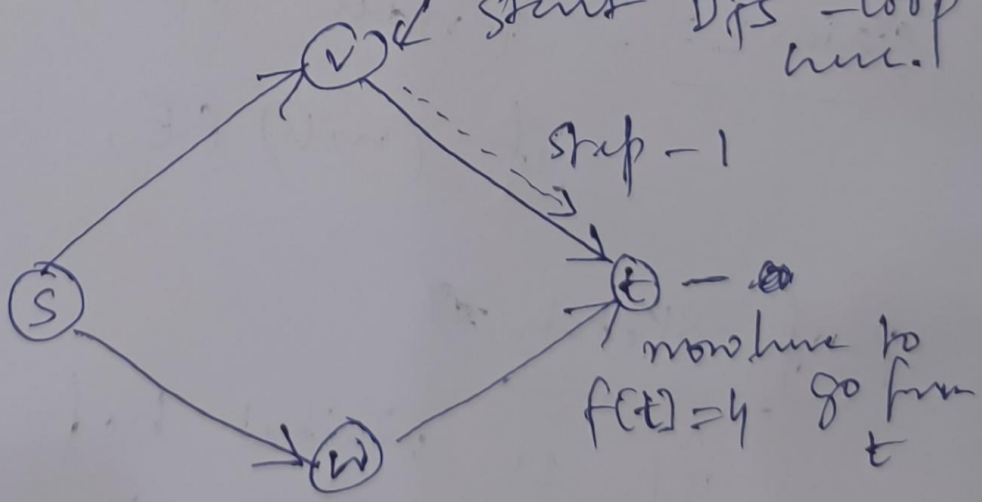
Topological ordering via DFS

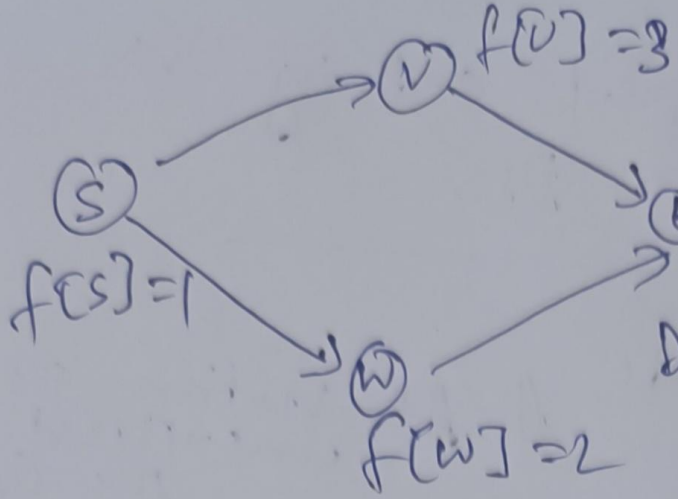
DFS pseudocode

// All vertices are unexplored before the call.

```
DFS (G,s)
    mark s as Explored
    for each (s,v) in adj list of s
        if v is unexplored
            DFS (G,v).
    set f(s) = Count_label.
    decrease Count label by 1.
```

Simulation





DFS would Consider t again But "explored" - So skip.

Next Consider s

for s — Consider v - explored.

w - unexplored

eventually,

DFS runs out of vertices ⇒ done!

Claim 1:- DFS-loop algorithm runs in $O(m+n)$ time.

Claim 2:- Under DFS loop,

if $(u,v) \in E$, then $f[u] < f[v]$

Proof Sketch ⇒

If u visited before v ⇒

Recursive call for v finishes before that of u

u's label < v's label

If v visited before u ⇒

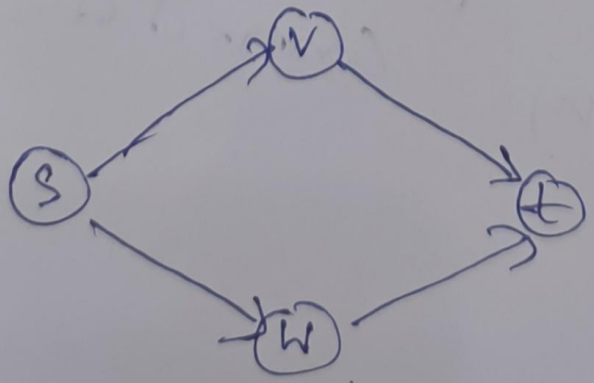
~~NO~~ NO v ⇝ u path (no cycle!).

⇒ DFS(v) won't discover u

⇒ v's recursive call finishes before u's

Note:- DFS-loop algorithm terminates on any i/p graph. (not necessarily a DAG) and returns some labeling f. (not topological ordering)

Strongly Connected Components



Connected?

Yes — if one looks at underlying undirected graph.

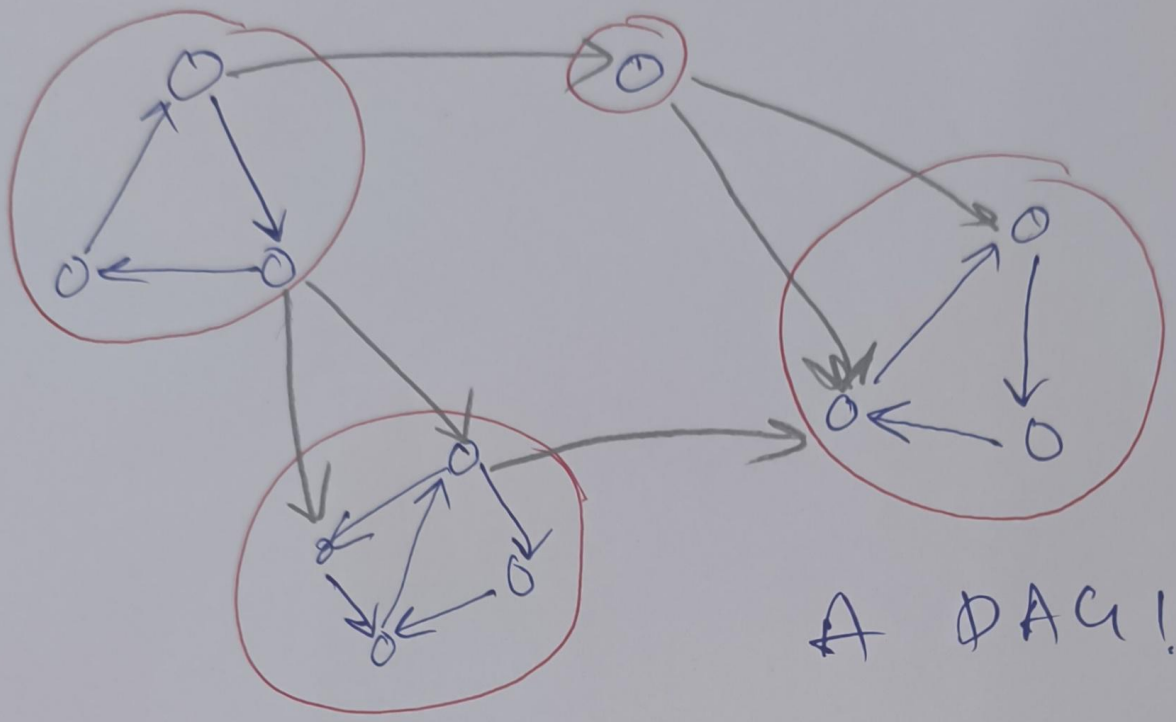
No — if one cares about reachability via directed edges.

Strongly Connected

A directed graph is Strongly Connected if every vertex can be reached from every other vertex by a directed path.

Strongly Connected Components

Maximal subgraph of G that is Strongly Connected.



A DAG!